

ATELIER C++



```
#include <iostream>
```

```
int main() {  
    std::cout << "Hello World!" << std::endl;  
    return 0;  
}
```

Notion des classes

1. Qu'est-ce qu'une classe ?

Une **classe** est un **modèle** ou un **plan** qui décrit un type d'objet.

Elle **regroupe** :

- des **données** (appelées *attributs* ou *membres de données*),
- et des **fonctions** (appelées *méthodes* ou *fonctions membres*).

👉 En résumé, **une classe = un modèle d'objet**, tandis qu'un **objet = une instance concrète** de cette classe.

Notion des classes

```
class Voiture {  
public: // zone accessible de l'extérieur  
    string marque;  
    int annee;  
  
    void demarrer() {  
        cout << "La voiture démarre " << endl;  
    }  
};
```

Nom de la classe

Visibilité des attributs et des méthodes

Attributs de la classe

Méthode de la classe

Notion des classes

- Une fois la classe définie, on peut **créer un objet** (ou instance) :

```
int main() {  
    Voiture v1;           // création d'un objet de type Voiture  
    v1.marque = "Toyota";  
    v1.annee = 2022;  
  
    cout << "Marque : " << v1.marque << endl;  
    cout << "Année : " << v1.annee << endl;  
    v1.demarrer();  
  
    return 0;  
}
```

Définition externe d'une classe

- Pour une meilleure représentation d'une classe en C++, on peut définir un prototype de la classe dans un **fichier .h** et la définition de ses méthodes dans un **autre fichier .cpp**

- **Fichier d'en-tête (.h) :**

Contient la déclaration de la classe, c'est-à-dire : les noms des attributs et des méthodes, les signatures des fonctions membres (sans leur corps). 👉 Ce fichier sert d'interface entre le programme et la classe.

- **Fichier source (.cpp) :**

Contient la définition des méthodes déclarées dans le fichier .h.

Chaque méthode est précédée du nom de la classe et de l'opérateur :: (appelé opérateur de portée).

Définition externe d'une classe

Exemple

- Le fichier **Etudiant.h**

```
#ifndef ETUDIANT_H
#define ETUDIANT_H

#include <string>
using namespace std;

class Etudiant {
private:
    string nom;
    int age;

public:
    Etudiant(string n, int a);
    void afficher();
};

#endif
```

On remarque l'utilisation des directives de compilation **#ifndef**, **#define** et **#endif** pour gérer les inclusions multiples du fichier header.

Attributs de la classe Etudiant

Méthode de la classe Etudiant

Définition externe d'une classe

Exemple

- Le fichier **Etudiant.cpp**

```
#include "Etudiant.h"
#include <iostream>
using namespace std;

Etudiant::Etudiant(string n, int a) {
    nom = n;
    age = a;
}

void Etudiant::afficher() {
    cout << "Nom : " << nom << ", Âge : " << age << endl;
}
```

Appel du Fichier **Etudiant.h**

Définition du
constructeur de la classe
Etudiant

Définition de la méthode
déclarée dans le fichier
Etudiant.h

Définition externe d'une classe

Exemple

- Le fichier **main.cpp**

```
#include "Etudiant.h"
```

Permet d'utiliser la classe Etudiant

```
int main() {  
    Etudiant e1("Amani", 25);  
    e1.afficher();  
    return 0;  
}
```

Création d'un objet e1 de type Etudiant et initialisation de ses attributs

Exécution d'une méthode sur l'objet e1

Notion des classes - This

- **En cas d'ambiguïté de nom** entre les attributs et d'autres variables dans les méthodes, on dispose du pointeur sur l'instance courante, `this`, qui nous aide à pointer les attributs de la classe.

```
void setLargeur(double largeur)
{
    this->largeur = largeur;
}
void setHauteur(double hauteur)
{
    this->hauteur = hauteur;
}
```

Constructeurs

- Les constructeurs sont des méthodes particulières qui ont pour responsabilité d'initialiser les attributs de la classe.
- Le constructeur porte le même nom que la classe, n'a pas de type de retour et est invoqué automatiquement où on crée une instance.
- Une classe peut avoir plusieurs constructeurs
- Si aucun constructeur n'est spécifié le compilateur fournit un constructeur par défaut à la classe (attention : laisse non initialisé les attributs de type de base).

Fichier Etudiant.h

```
#ifndef ETUDIANT_H
#define ETUDIANT_H

#include <string>
using namespace std;

class Etudiant
{
public:
    // Les constructeurs
    Etudiant(string n, int a);
    Etudiant();

    // Les méthodes
    string getNom() const;
    int getAge() const;
    void afficher() const;

private:
    string nom;
    int age;
};

#endif
```

Fichier Etudiant.cpp

```
#include "Etudiant.h"
#include <iostream>
using namespace std;

// constructeur avec paramètres
Etudiant::Etudiant(string n, int a)
{
    this->nom = n;
    this->age = a;
}

// constructeur par défaut
Etudiant::Etudiant()
{
    this->nom = "";
    this->age = 0;
}

// méthodes
string Etudiant::getNom() const
{
    return this->nom;
}
```


Constructeurs – Liste d'initialisation

Les listes d'initialisation servent pour initialiser les attributs et/ou d'appeler leurs constructeurs (s'ils sont des objets).

Constructeurs SANS liste d'initialisation

```
#include "Etudiant.h"

Etudiant::Etudiant(string n, int a)
{
    this->nom = n;
    this->age = a;
}

Etudiant::Etudiant()
{
    this->nom = "";
    this->age = 0;
}
```

Constructeurs AVEC liste d'initialisation

```
Etudiant::Etudiant(string n, int a)
    : nom(n), age(a)
{
}

Etudiant::Etudiant()
    : nom(""), age(0)
{
}
```

Avec valeur par défaut

```
Etudiant::Etudiant(string n = "", int a = 0)
    : nom(n), age(a)
{
}
```

Constructeurs par défaut

```
class Etudiant {  
private:  
    string nom;  
    int age;  
    // suite...  
};
```

Constructeur par défaut implicite

→ Aucun constructeur n'est défini → le compilateur génère **un constructeur par défaut implicite**, mais **les attributs ne sont pas initialisés**.

```
class Etudiant {  
private:  
    string nom;  
    int age;  
  
public:  
    Etudiant()  
        : nom(""), age(0)  
    {}  
    // suite...  
};
```

Constructeur par défaut qui initialise les attributs à 0

→ Constructeur par défaut explicite qui initialise nom à " " age à 0

Constructeurs par défaut

```
class Etudiant {  
private:  
    string nom;  
    int age;  
  
public:  
    Etudiant(string n = "", int a = 0)  
        : nom(n), age(a)  
    {}  
    // suite...  
};
```

Constructeur avec paramètres par défaut

- Ici, **un seul constructeur**, mais il joue à la fois :
- le rôle de **constructeur paramétré**,
 - et de **constructeur par défaut** (si on appelle `Etudiant e;`
→ utilise `n=""` et `a=0`).

Constructeurs par défaut

```
class Etudiant {  
private:  
    string nom;  
    int age;  
  
public:  
    Etudiant(string n, int a)  
        : nom(n), age(a)  
    {}  
    // suite...  
};
```

Pas de constructeur par défaut

- Comme il existe un constructeur paramétré **sans valeurs par défaut**,
- le compilateur **ne génère pas** de constructeur par défaut.
- Donc `Etudiant e;` n'est pas possible ❌

Exercice

- On désire représenter l'objet Point caractérisé par un abscisse et un ordonné de type réel. Un point est construit de deux manières:
 - Les coordonnées sont par défaut à 0
 - Les coordonnées sont initialisés par des valeurs utilisateurs
- Un point devrait être afficher sous cette forme (1.3 , 2.9)
- Un point peut être déplacer selon l'axe des abscisses et/ou l'axe des ordonnés.
- Définir la classe point.
- Ecrire une fonction main qui crée un point A à l'origine du repère. Puis on crée un autre point B ayant des coordonnées saisis au clavier. Afficher les deux points.
- En fin, faire confondre les deux points et les afficher de nouveau.

Exercice – Correction

```
class point
{
private:
    double abs;
    double ord;
public:
    point(double ab=0,double o=0);
    void deplacer(double, double) ;
    void afficher() const;
    double get_abs() const;
    double get_ord() const;
    void set_abs(double);
    void set_ord(double);
};
```

```
#include<iostream>
#include "point.h"

point::point(double ab, double o):abs(ab),ord(o){}
void point::deplacer(double x, double y)
{
    this->abs += x;
    this->ord += y;
}
void point::afficher() const
{
    std::cout << "(" << abs << " , " << ord << ")\n";
}
double point::get_abs() const
{
    return abs;
}
double point::get_ord() const
{
    return ord;
}
void point::set_abs(double abs)
{
    this->abs = abs;
}
void point::set_ord(double ord)
{
    this->ord = ord;
}
```


Exercice – Correction

```
#include <iostream>
#include "point.h"

int main()
{
    double x, y;
    point A;
    std::cout << "Donner les coordonnes de B:";
    std::cin >> x;
    std::cin >> y;

    point B(x,y);
    std::cout << "Le point A:";
    A.afficher();
    std::cout << "Le point B:";
    B.afficher();
    std::cout << "confondre A et B ....\n";
    A.set_abs(B.get_abs());
    A.set_ord(B.get_ord());
    std::cout << "Le point A:";
    A.afficher();
    std::cout << "Le point B:";
    B.afficher();
}
```

Constructeur de copie

Un constructeur de copie est appelé lors de l'instanciation d'un objet avec en argument d'appel un objet du même type. Il est aussi appelé lors du passage par valeur d'un objet en paramètre, ainsi que lors du retour d'une fonction ou méthode qui retourne un objet de ce type. Le rôle d'un constructeur par copie est de permettre l'instanciation d'un nouvel objet dans le même état qu'un objet existant (ou clone).

Syntaxe :

fichier Toto.h

```
class Toto {  
    ...  
    Toto( Toto & autre );  
    ...  
};
```

fichier Toto.cxx

```
#include "Toto.h"  
...  
Toto::Toto( Toto & autre )  
{ ... }
```


Constructeur de copie

Ce genre de constructeur permet d'initialiser un objet avec un autre objet.

Exemple classe Etudiant :

```
Etudiant::Etudiant(Etudiant const& e)
    : nom(e.nom), age(e.age)
{
}
```

Le compilateur crée automatiquement un constructeur de copie si le développeur n'en fournit pas.

Ce constructeur de copie par défaut copie chaque attribut de l'objet, exactement comme le ferait un constructeur de copie écrit manuellement.

C'est pourquoi, dans la plupart des cas, on utilise simplement celui généré par le compilateur et on n'a pas besoin de définir un constructeur de copie nous-même.

Destructeur d'une classe

Le **destructeur** en C++ est une fonction spéciale (méthode particulière) qui s'exécute **automatiquement** quand un objet est **détruit**.

C'est l'opposé du **constructeur**.

Syntaxe du destructeur

Il porte le même nom que la classe, précédé d'un **tilde (~)**, et n'a aucun paramètre.

```
class Point {  
public:  
    ~Point() {  
        // Code exécuté à la destruction  
    }  
};
```

- Pas de valeur de retour
- Pas de paramètres
- Un seul destructeur par classe

Destructeur d'une classe

Le destructeur en C++ sert à **libérer les ressources** utilisées par l'objet :

- libérer de la mémoire (delete, free)
- libérer un tableau dynamique (delete[])
- afficher un message de destruction (pour tests)

```
class Etudiant {  
private:  
    int* notes;  
public:  
    Etudiant() {  
        notes = new int[10]; // allocation  
    }  
  
    ~Etudiant() {  
        delete[] notes;      // libération  
        cout << "Etudiant supprimé" << endl;  
    }  
};
```


Destructeur d'une classe

A quel moment le constructeur s'exécute ?

1) Un objet local (automatique) sort de sa portée :

Quand un objet est déclaré à l'intérieur d'un bloc { }, il est détruit automatiquement à la fin de ce bloc.

```
{  
    Point A; // créé ici  
}           // A est détruit ici → destructeur appelé
```

➡ Dès qu'on sort du bloc, le destructeur s'exécute.

2) Un objet créé avec new est détruit avec delete

Les objets créés avec new ne sont **pas détruits automatiquement**.

```
Point* p = new Point();  
delete p; // destructeur appelé ici
```

➡ Il faut appeler delete → sinon fuite mémoire.

Destructeur d'une classe

3) Un objet temporaire prend fin

Un objet temporaire est créé par le compilateur, souvent dans une expression.

Exemple :

```
Point temp = Point(3,4);
```

```
// destructeur de temp appelé juste après utilisation
```

Ou :

```
afficherPoint(Point(2,5)); // le Point(2,5) temporaire meurt après l'appel
```

➔ **Le destructeur s'exécute juste après l'utilisation du temporaire.**

4) Le programme se termine

À la fin du programme, tous les objets **globaux** ou **statiques** sont détruits.

```
static Point P; // créé au lancement, détruit à la fin du programme
```

➔ **Le destructeur est appelé automatiquement quand le programme quitte.**

Surcharge d'opérateur

La surcharge d'opérateur en C++ permet de redéfinir le comportement des opérateurs intégrés (comme +, -, ==, <<) pour qu'ils fonctionnent avec des classes personnalisées, rendant le code plus intuitif, comme si les objets étaient des types primitifs.

Cela se fait en définissant une fonction spéciale nommée `operator@` (où @ est l'opérateur), soit comme fonction membre de la classe, soit comme fonction globale ou amie, en expliquant au compilateur comment l'opération doit s'appliquer aux objets. Le nombre d'arguments dépend si l'opérateur est unaire (un argument) ou binaire (deux arguments).

Surcharge d'opérateur

Exemple : Soit la classe Etudiant suivante

```
class Etudiant {  
private:  
    std::string nom;  
    float note;  
  
public:  
    Etudiant(std::string n, float no) : nom(n), note(no) {}  
  
    float getNote() const { return note; }  
    std::string getNom() const { return nom; }  
};
```

➤ **Surcharge de l'opérateur +:**

On va additionner les notes de deux étudiants e1 et e2

```
Etudiant operator+(const Etudiant& e1, const Etudiant& e2) {  
    return Etudiant(e1.getNom() + " & " + e2.getNom(), e1.getNote() + e2.getNote());  
}
```

Exemple :

```
Etudiant a("Ahmed", 12.5);  
Etudiant b("Maryam", 15.0);  
  
Etudiant c = a + b;
```

Etudiant c aura :

- nom = « Ahmed & Maryam »
- note = 27.5

Surcharge d'opérateur

Exemple : Soit la classe Etudiant suivante

```
class Etudiant {  
private:  
    std::string nom;  
    float note;  
  
public:  
    Etudiant(std::string n, float no) : nom(n), note(no) {}  
  
    float getNote() const { return note; }  
    std::string getNom() const { return nom; }  
};
```

➤ Surcharge de l'opérateur == :

On va comparer deux étudiants par note :

```
bool operator==(const Etudiant& e1, const Etudiant& e2) {  
    return e1.getNote() == e2.getNote();  
}
```

➤ Surcharge de l'opérateur < (Comparaison)

```
bool operator<(const Etudiant& e1, const Etudiant& e2) {  
    return e1.getNote() < e2.getNote();  
}
```

Surcharge d'opérateur

Exemple : Soit la classe Etudiant suivante

```
class Etudiant {  
private:  
    std::string nom;  
    float note;  
  
public:  
    Etudiant(std::string n, float no) : nom(n), note(no) {}  
  
    float getNote() const { return note; }  
    std::string getNom() const { return nom; }  
};
```

➤ **Surcharge de l'opérateur << (affichage) :**

C'est le plus utilisé : afficher un étudiant avec cout <<

```
#include <iostream>  
  
std::ostream& operator<<(std::ostream& out, const Etudiant& e) {  
    out << "Nom: " << e.getNom() << ", Note: " << e.getNote();  
    return out;  
}
```


Exemple : Soit la classe Etudiant suivante

```
#include <iostream>
#include <string>
using namespace std;

class Etudiant {
private:
    string nom;
    float note;

public:
    Etudiant(string n = "", float no = 0) : nom(n), note(no) {}

    float getNote() const { return note; }
    string getNom() const { return nom; }
};
```

```
// Addition des notes
Etudiant operator+(const Etudiant& e1, const Etudiant& e2) {
    return Etudiant(e1.getNom() + " & " + e2.getNom(), e1.getNote() + e2.getNote());
}

// Comparaison ==
bool operator==(const Etudiant& e1, const Etudiant& e2) {
    return e1.getNote() == e2.getNote();
}

// Affichage <<
ostream& operator<<(ostream& out, const Etudiant& e) {
    out << "Nom: " << e.getNom() << ", Note: " << e.getNote();
    return out;
}

int main() {
    Etudiant a("Ahmed", 12.5);
    Etudiant b("Maryam", 15.0);

    cout << a << endl;
    cout << b << endl;

    Etudiant c = a + b;
    cout << c << endl;

    if (a == b)
        cout << "Même note" << endl;
}
```

Surcharge Interne - Surcharge externe

✓ Surcharge interne (méthode membre)

La surcharge interne signifie que l'opérateur est défini *à l'intérieur de la classe*, comme une *méthode membre*.

Caractéristiques :

- L'opérateur est dans la classe
- Il a accès directement aux attributs privés
- Le premier opérande est toujours l'objet courant (`this`)

```
class Etudiant {  
private:  
    string nom;  
    float note;  
  
public:  
    Etudiant(string n="", float no=0) : nom(n), note(no) {}  
  
    // surcharge interne  
    Etudiant operator+(const Etudiant& e) const {  
        return Etudiant(nom + " & " + e.nom, note + e.note);  
    }  
};
```

